

test-quad例程

将如下文件保存为filament/samples/test-quad.cpp

```
#include <cstdint>
#include <cstring>

#include <filament/Camera.h>
#include <filament/Engine.h>
#include <filament/Material.h>
#include <filament/MaterialInstance.h>
#include <filament/FilamentAPI.h>
#include <filament/LightManager.h>
#include <filament/RenderableManager.h>
#include <filament/Scene.h>
#include <filament/View.h>
#include <filament/Skybox.h>
#include <filament/Viewport.h>
#include <filament/Renderer.h>
#include <filament/VertexBuffer.h>
#include <filament/IndexBuffer.h>
#include <utils/EntityManager.h>
#include <filament/TransformManager.h>
#include <math/norm.h>

#include <SDL.h>

#include <thread>
#include <string>
#include <vector>
#include <fstream>
#include <iostream>
// #include "generated/resources/resources.h"

using namespace filament;
using namespace utils;
using namespace math;
using namespace std;

#include <utils/Panic.h>
#include <SDL_syswm.h>

SDL_Window* createSDLwindow() {

    const uint32_t windowFlags =
        SDL_WINDOW_SHOWN | SDL_WINDOW_RESIZABLE | SDL_WINDOW_ALLOW_HIGHDPI;
    SDL_Window* win =
        SDL_CreateWindow("Hello World!", 100, 100, 1920, 1080, windowFlags);
    if (win == nullptr) {
        std::cout << "SDL_CreateWindow Error: " << SDL_GetError() << std::endl;
        SDL_Quit();
        return NULL;
    }

    return win;
}

void* getNativeWindow(SDL_Window* sdlWindow) {
    SDL_SysWMInfo wmi;
    SDL_VERSION(&wmi.version);
    ASSERT_POSTCONDITION(SDL_GetWindowWMInfo(sdlWindow, &wmi), "SDL version unsupported!");
    if (wmi.subsystem == SDL_SYSWM_X11) {
        #if defined(FILAMENT_SUPPORTS_X11)
            Window win = (Window) wmi.info.x11.window;
            return (void *) win;
        #endif
    }
    else if (wmi.subsystem == SDL_SYSWM_WAYLAND) {
        #if defined(FILAMENT_SUPPORTS_WAYLAND)
            int width = 0;

```

```

        int height = 0;
        SDL_GetWindowSize(sdlWindow, &width, &height);

        // Static is used here to allocate the struct pointer for the lifetime of the program.
        // Without static the valid struct quickly goes out of scope, and ends with seemingly
        // random segfaults.
        static struct {
            struct wl_display *display;
            struct wl_surface *surface;
            uint32_t width;
            uint32_t height;
        } wayland {
            wmi.info.wl.display,
            wmi.info.wl.surface,
            static_cast<uint32_t>(width),
            static_cast<uint32_t>(height)
        };
        return (void *) &wayland;
#endif
    }
    return nullptr;
}

std::ifstream::pos_type getFileSize(const char* filename) {
    std::ifstream in(filename, std::ifstream::ate | std::ifstream::binary);
    return in.tellg();
}

int main(int argc, char** argv) {
    float3 vertices[] = {
        { -10, 0, -10 },
        { -10, 0, 10 },
        { 10, 0, 10 },
        { 10, 0, -10 },
    };

    const static uint32_t indices[] = { 0, 1, 2, 2, 3, 0 };

    short4 tbn = math::packSnorm16(
        mat3f::packTangentFrame(math::mat3f{ float3{ 1.0f, 0.0f, 0.0f },
            float3{ 0.0f, 0.0f, 1.0f },
            float3{ 0.0f, 1.0f, 0.0f } })
        .xyzw);

    const static math::short4 normals[] { tbn, tbn, tbn, tbn };
    SDL_Window* window = createSDLwindow();
    Engine* engine = Engine::create();
    SwapChain* swapChain = engine->createSwapChain(getNativeWindow(window));
    Renderer* renderer = engine->createRenderer();

    TransformManager& tcm = engine->getTransformManager();

    Entity cameraEntity = utils::EntityManager::get().create();
    Camera* camera = engine->createCamera(cameraEntity);
    View* view = engine->createView();
    Scene* scene = engine->createScene();

    view->setCamera(camera);
    // Determine the current size of the window in physical pixels.
    uint32_t w, h;
    SDL_GL_GetDrawableSize(window, (int*)&w, (int*)&h);

    camera->setProjection(45, 16.0/9.0, 0.1, 1.0);
    //camera->setProjection(45.0, double(w) / h, 0.1, 50);//, Camera::Fov::VERTICAL);
    view->setViewport({ 0, 0, w, h });

    view->setScene(scene);

    VertexBuffer* vertexBuffer =
        VertexBuffer::Builder()

```

```

        .vertexCount(4)
        .bufferCount(2)
        .attribute(VertexAttribute::POSITION, 0,
                    VertexBuffer::AttributeType::FLOAT3)
        .attribute(VertexAttribute::TANGENTS, 1,
                    VertexBuffer::AttributeType::SHORT4)
        .normalized(VertexAttribute::TANGENTS)
        .build(*engine);

vertexBuffer->setBufferAt(
    *engine, 0,
    VertexBuffer::BufferDescriptor(vertices, vertexBuffer->getVertexCount() * sizeof
(vertices[0])));
vertexBuffer->setBufferAt(
    *engine, 1,
    VertexBuffer::BufferDescriptor(normals, vertexBuffer->getVertexCount() * sizeof(normals
[0])));

IndexBuffer* indexBuffer =
    IndexBuffer::Builder().indexCount(6).build(*engine);

indexBuffer->setBuffer(
    *engine, IndexBuffer::BufferDescriptor(indices, indexBuffer->getIndexCount() * sizeof
(uint32_t)));

std::string filename = "generated/material/aiDefaultMat.filamat";
//std::string filename = "generated/material/image.filamat";
long fileSize = static_cast<long>(getFileSize(filename.c_str()));
if (fileSize <= 0) {
    return 1;
}

Material* material = nullptr;
MaterialInstance* materialInstance = nullptr;
std::ifstream in(filename.c_str(), std::ifstream::in);
if (in.is_open()) {

    std::vector<char> materialBuffer;
    materialBuffer.reserve(static_cast<unsigned long>(fileSize));

    in.read(materialBuffer.data(), fileSize);
    material = Material::Builder()
        .package((void*)materialBuffer.data(), (size_t)fileSize)
        .build(*engine);
    materialInstance = material->createInstance();

}

materialInstance->setParameter("baseColor", float3{ 0, 0.8, 0.8 });
materialInstance->setParameter("metallic", 0.0f);
materialInstance->setParameter("roughness", 0.4f);
materialInstance->setParameter("reflectance", 0.5f);

Entity renderable = EntityManager::get().create();

Entity light = EntityManager::get().create();

LightManager::Builder(LightManager::Type::SUN)
    .color(Color::toLinear<ACCURATE>(sRGBColor(0.98f, 0.92f, 0.89f)))
    .intensity(110000)
    .direction({ 0.7, -1, -0.8 })
    .sunAngularRadius(1.9f)
    .castShadows(true)
    .build(*engine, light);

scene->addEntity(light);

// build a quad
RenderableManager::Builder(1)
    .boundingBox({ { -1, -1, -1 }, { 1, 1, 1 } })

```

```

        .material(0, materialInstance)
        .geometry(0, RenderableManager::PrimitiveType::TRIANGLES, vertexBuffer,
                  indexBuffer, 0, 6)
        .culling(false)
        .build(*engine, renderable);

    scene->addEntity(renderable);
    mat4 origTransform = math::mat4::translation(double3{ 0, -1, -100 }) * mat4::rotation(0.5 * M_PI,
double3{ 1, 0, 0 });
    mat4 camTrans = camera->getModelMatrix();
    tcm.setTransform(tcm.getInstance(renderable), origTransform);

    constexpr double speed = 100.0;
    bool run = true;
    double g_discoAngle = 0;
    double g_discoAngularSpeed = 1;
    double lastTime = 0;

    float3 pos = 0;
    while (run) {
        // beginFrame() returns false if we need to skip a frame
        if (renderer->beginFrame(swapChain)) {
            // for each View
            renderer->render(view);
            renderer->endFrame();
        }
        double now = (double)SDL_GetPerformanceCounter() / SDL_GetPerformanceFrequency();
        double dT = now - lastTime;
        SDL_Event event;
        int numEvents = 0;
        while (SDL_PollEvent(&event) && numEvents < 16) {

            switch (event.type) {
                case SDL_QUIT:
                    run = false;
                    break;
            }
            numEvents++;
        }

        SDL_Delay(16);
        mat4f transform = mat4f::translation(float3{ 0, -1, -50 }) * mat4f::rotation(g_discoAngle,
float3{ 0, 0, 1 });
        transform *= origTransform;
        //tcm.setTransform(tcm.getInstance(renderable), transform);
        g_discoAngle += g_discoAngularSpeed * dT;

        lastTime = now;
    }

    Engine::destroy(&engine);

    return 0;
}

```

在add_demo部分后面补充对应的选项:

```

add_demo(viewtest)
add_demo(test-quad)

```

编译即可运行。

目前只在ubuntu 20.04测试通过。